```
class MaternGPModel(gpytorch.models.ExactGP):
    def __init__(self, train_x, train_y, likelihood, nu):
        super().__init__(train_x, train_y, likelihood)
        self.mean_module = gpytorch.means.ZeroMean()
        self.covar_module = gpytorch.kernels.MaternKernel(nu)

    def forward(self, x):
        ...  (1)
```

(1) Omitted.

Here, our `covar_module` attribute is initialized as an instance of the `gpytorch.kernels.MaternKernel` class. This initialization takes in a parameter `nu` that defines the level of smoothness that our GP will have, which is also a parameter of our `__init__()` method.

IMPORTANT **At the time of this writing, three values for `nu` are supported by GPyTorch: 0.5, 1.5, and 2.5, corresponding to functions being non-, once-, and twice-differentiable. In other words, the larger this `nu` parameter is, the smoother our GP is.**

Let's try `nu = 0.5` first by setting that value when we initialize the GP:

```
likelihood = gpytorch.likelihoods.GaussianLikelihood()
model = MaternGPModel(train_x, train_y, likelihood, 0.5)

...  (1)

visualize_gp_belief(model, likelihood)
```

(1) Fixing hyperparameters, enabling prediction model.

This code produces Figure 3.13.

©Manning Publications Co. To comment go to liveBook
https://livebook.manning.com/#!/book/bayesian-optimization-in-action/discussion

Licensed to ashwin sampath <ashwin.manning@gmail.com>